

Assignment-2 | October 2021

Branch Prediction

ELL 782 – Computer Architecture

Submitted by:

Aniruddh Nagar (2021EET2113)

1. Introduction:

What is Branch Prediction?

In computer architecture, Branch prediction is a technique used in modern CPU designs that attempts to guess the outcome of a conditional operation and prepare the outcome for the most likely to meet.

A digital circuit that tries to guess which way a branch will go before this is known definitively is known as a branch predictor. The purpose of the branch predictor is to *counter the fetch limitation imposed by control hazards* and improve the flow in the instruction pipeline and hence, achieve high effective performance. When branch prediction rates are high enough to counter miss-prediction penalties, the processor is said to have a better overall performance. In absence of branch prediction, processors may stall whenever there is an unresolved branch instruction; this degrades the performance of the CPUs.

How does it work?

When a conditional operation (if...else) is to be processed, the branch predictor guesses which condition will be met most likely. Then, it executes the respective operations required by this guess ahead of time. In this way, the operations are already completed if the guess was correct. At runtime, if the guess turns out to be incorrect, the CPU executes the other corresponding branch of operation, causing a slight delay. Speed is significantly increased, in case the guess is correct. The more frequently the same operation is used, the more accurate the guess will become.

For predictor implementation in this assignment, I have used a 2- bit saturating counter which will work as follows:

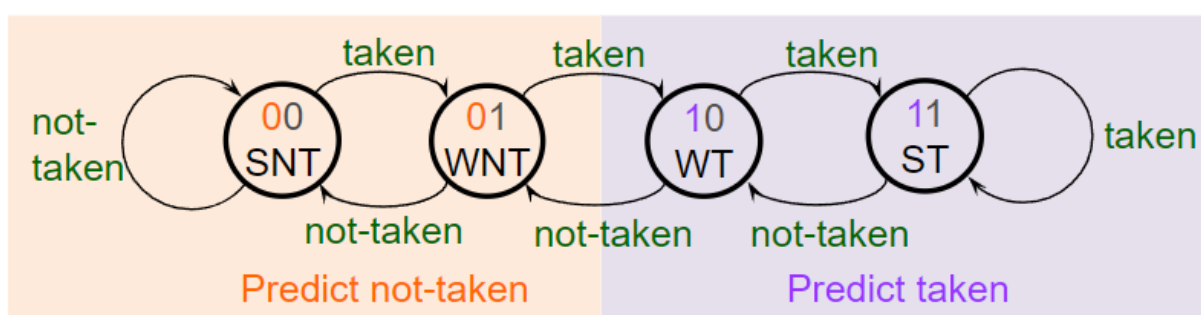


Figure 1: 2-bit Saturating Counter

This 2 bit saturating counter changes its prediction when there happen two consecutive incorrect predictions. It results in better prediction than normal saturating counters.

2. Branch Predictor Implementation:

2.1. Size: 2400 bits, Predictor: Gshare:

For the 2400 area budget, I have implemented a simple G-share predictor. The working of the G-share predictor is shown below:

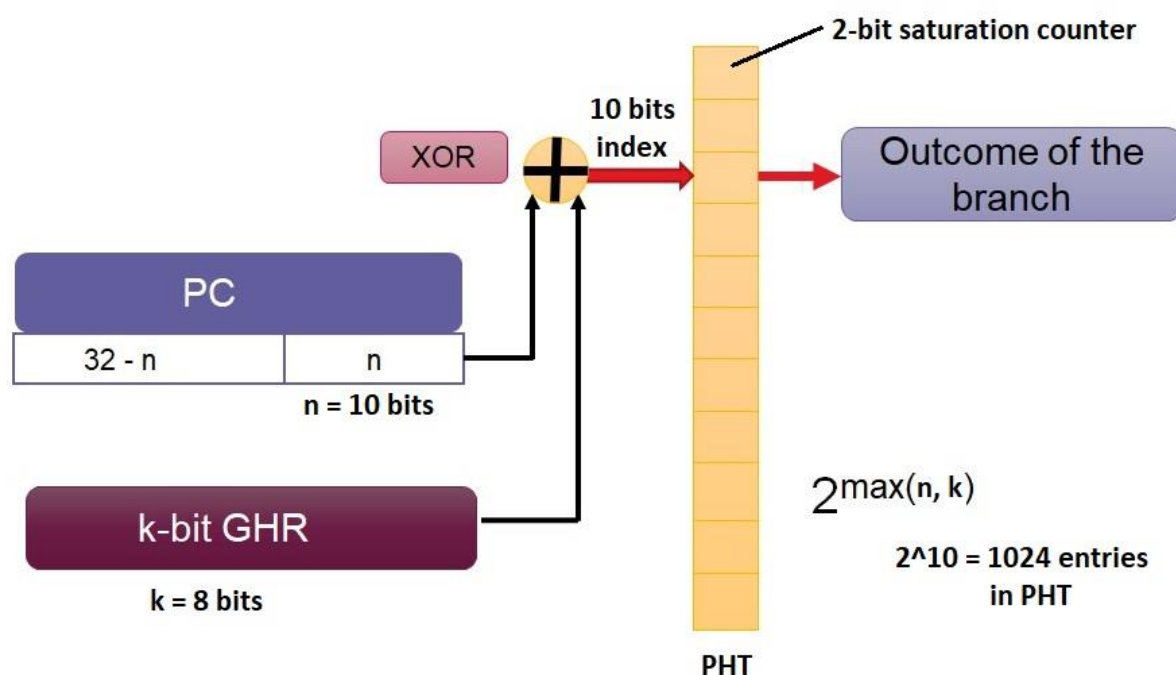


Figure 2: Gshare Predictor

In the Gshare predictor, Branch and pattern history are kept globally. Branch address i.e. (n=10) LSB-bits from PC and global history i.e. (k=8)bits from GHR are XORed and the result is used to index the PHT (1024x2)bits to access the 2 bit saturating counter.

Based on the outcomes, the train function will update entries in the PHT table. The mechanism shown in figure1 will be used to update saturating counter values for the next prediction. For the GHR register, the GHR register bits will be shifted left by 1 position and the current outcome is appended at the LSB bit of the GHR register.

Note: Due to adjustments, the accuracy outputs are provided in further pages.

2.2. Size: 6400 bits, Predictor: Gshare + PAp:

For the 6400 area budget, I have implemented a simple Tournament predictor. It selects the desired prediction from any of the Gshare and PAp predictors. The working of G-share has been seen earlier. The working of PAp & Tournament predictors is shown:

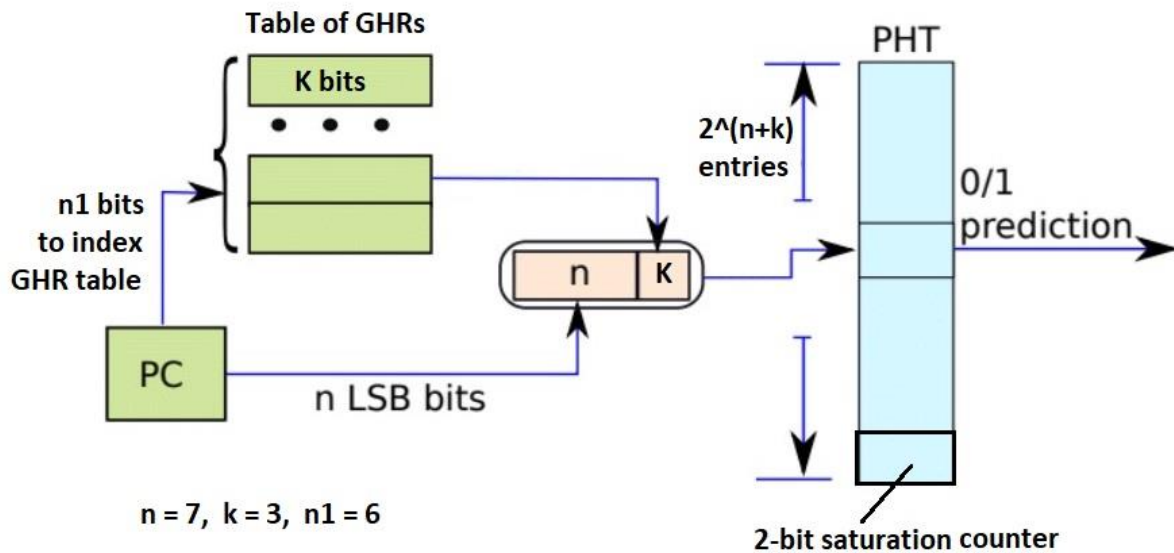


Figure 3: PAp Predictor

The above implementation has resulted in satisfactory accuracy. In this implementation, I have used a dedicated (1024 entries) PHT table, (3-bit GHR) registers and (64 entries) GHR table for PAp. For Gshare I have considered i.e. (n=10) LSB-bits as Branch address from PC and (k=8) bits global history from GHR.

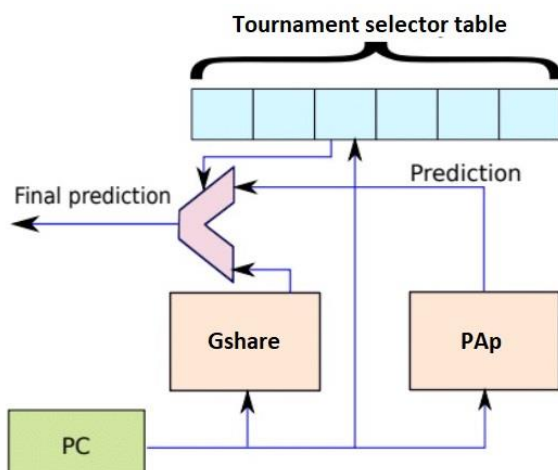


Figure 4: Tournament Predictor

For Tournament implementation, I have considered a selector table with (1024 entries) of 2 bits each and an 8-bit register.

This implementation uses both Gshare and PAp to predict the branch outcome.

It reads the corresponding entry in the selector table and based on that it chooses prediction from any one of the predictors.

To update the tournament selector table, first, a comparison is done between the final branch outcome and the individual predictions. Then based on comparison results the corresponding entry in the selector table is updated.

The Accuracy output for different traces of budget size 2400 & 6400 is shown below:

```

ruddh@LAPTOP-...: /mnt/c/Users/hp/CTech_
extracting...
DONE

-----
MAX SIZE = 2400
-----

file = Predictor2400

TRACE: trace1
size = 2056
accuracy = 0.920911083073

TRACE: trace2
size = 2056
accuracy = 0.974930208301

TRACE: trace3
size = 2056
accuracy = 0.987816112909

TRACE: trace4
size = 2056
accuracy = 0.988144114699

TRACE: trace5
size = 2056
accuracy = 0.93513426029

average predictor accuracy = 0.961387155854
expected average accuracy = 0.9484

```

Figure 5: Accuracy for predictor size 2400

```

ruddh@LAPTOP-...: /mnt/c/Users/hp/CTech_
-----
MAX SIZE = 6400
-----

file = Predictor6400

TRACE: trace1
size = 6344
accuracy = 0.943436090154

TRACE: trace2
size = 6344
accuracy = 0.980124216674

TRACE: trace3
size = 6344
accuracy = 0.990016789533

TRACE: trace4
size = 6344
accuracy = 0.989594147182

TRACE: trace5
size = 6344
accuracy = 0.977370812894

average predictor accuracy = 0.976108411288
expected average accuracy = 0.9513

```

Figure 6: Accuracy for predictor size 6400

The Accuracy output for different traces of budget size 2400 & 6400 is shown below:

Traces :	Accuracy:
Trace1	0.9209
Trace2	0.9749
Trace3	0.9878
Trace4	0.9881
Trace5	0.9351
Average	0.9613

Table 1: Accuracy for predictor size 2400

Traces :	Accuracy:
Trace1	0.9434
Trace2	0.9801
Trace3	0.9900
Trace4	0.9895
Trace5	0.9773
Average	0.9761

Table 2: Accuracy for predictor size 6400

2.3. Size: 9999 bits, Predictor: Global + Local:

For the 9999 area budget, I have used a tournament predictor with a combo of Local predictor and Gshare predictors. Implementation of such predictor is shown here. It has provided a satisfactory accuracy value for the assigned area budget.

In the Local predictor, I have considered a (512 entry) LHT-table with each entry size of 9 bits. This table is accessed by a 9-bit index from PC. The result value from this table is then used as an index to the PHT table. The value here denotes the prediction decision.

In the Gshare predictor, Branch and pattern history are kept globally. Branch address i.e. (n=10) LSB-bits from PC and global history i.e. (k=10)bits from GHR are XORed and the result is used to index the PHT (1024x2)bits to access the 2 bit saturating counter

For prediction selection, I have considered a (1024-entry) select table with each entry size of 2 bits. To access these entries (10 bits) from PC have been used as an index.

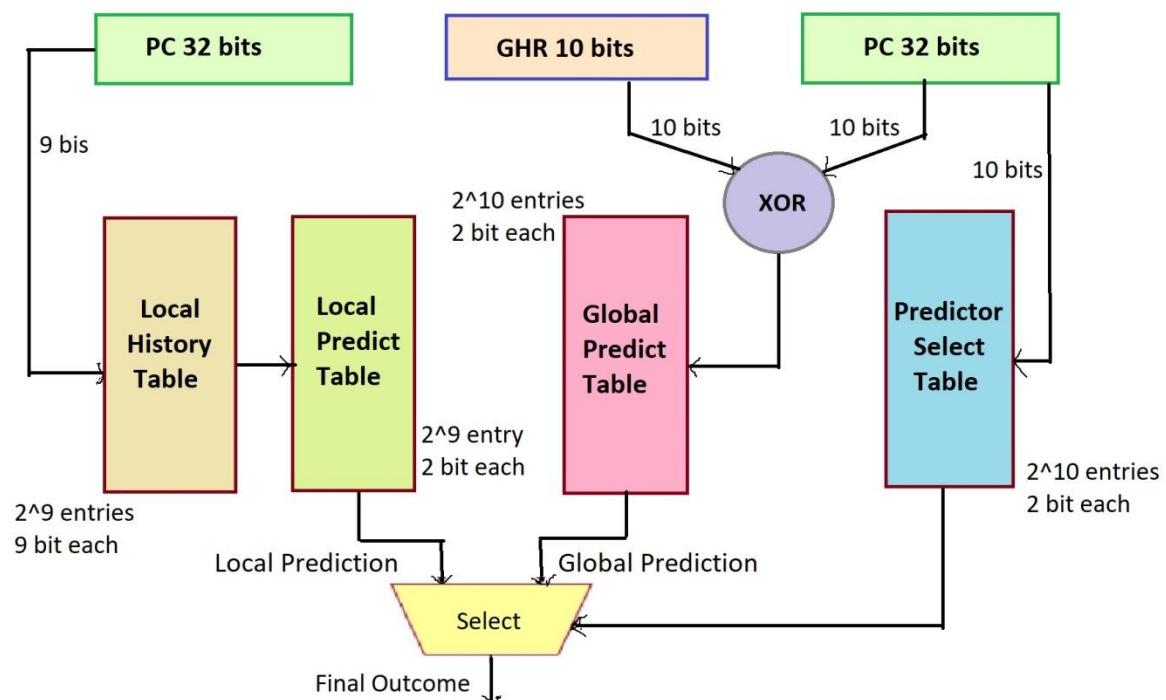


Figure 7: Tournament Predictor (Gshare + Local) : 9999

To update predict table for LHT and predict table for Gshare the true branch outcome is used. The update is done by the 2bit saturation counter shown in figure1. In the case of a select table, if the true outcome is similar to Gshare prediction then decrease saturation counter, if the true outcome is similar to Local predictor prediction then increase saturation counter and if the prediction of both predictors is similar then no update is needed.

2.4. Size: 32000 bits, Predictor: Global + Local:

For the 32000 area budget, I have used a tournament predictor with a combo of Local predictor and Gshare predictors. Implementation of such predictor is shown here. It has provided a satisfactory accuracy value for the assigned area budget.

In the Local predictor, I have considered a (1024 entry) LHT-table with each entry size of 11 bits. This table is accessed by a 10-bit index from PC. The result value from this table is then used as an index to the PHT table. The value here denotes the prediction decision.

In the Gshare predictor, Branch and pattern history are kept globally. Branch address i.e. (n=12) LSB-bits from PC and global history i.e. (k=12)bits from GHR are XORed and the result is used to index the PHT (4096x2)bits to access the 2 bit saturating counter

For prediction selection, I have considered a (4096-entry) select table with each entry size of 2 bits. To access these entries the same XOR result is used which is generated for GShare predictor.

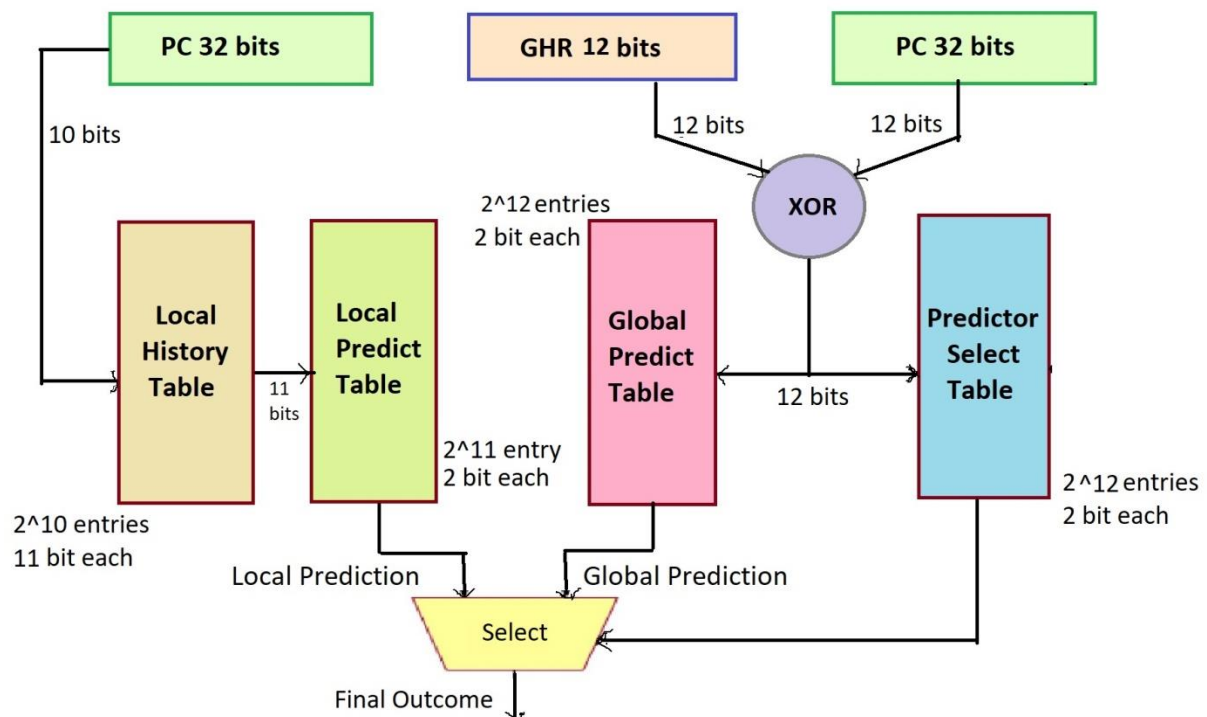


Figure 8: Tournament Predictor (Gshare + Local) : 32000

To update predict table for LHT and predict table for Gshare the true branch outcome is used. The update is done by the 2bit saturation counter shown in figure1. In the case of a select table, if the true outcome is similar to Gshare prediction then decrease saturation counter, if the true outcome is similar to Local predictor prediction then increase saturation counter and if the prediction of both predictors is similar then no update is needed.

The Accuracy output for different traces of budget size 9999 & 32000 is shown below:

```

ruddh@LAPTOP-...:~/mi x + v
MAX SIZE = 9999
-----

file = Predictor9999

TRACE: trace1
size = 9770
accuracy = 0.959482723961

TRACE: trace2
size = 9770
accuracy = 0.972812891315

TRACE: trace3
size = 9770
accuracy = 0.990039416937

TRACE: trace4
size = 9770
accuracy = 0.988327182695

TRACE: trace5
size = 9770
accuracy = 0.984830199554

average predictor accuracy = 0.979098482892
expected average accuracy = 0.9531
ruddh@LAPTOP-...:~/mnt/c/Users/hp/CTech_

```

Figure 9: Accuracy for predictor size 9999

```

ruddh@LAPTOP-...:~/mi x + v
MAX SIZE = 32000
-----

file = Predictor32000

TRACE: trace1
size = 31788
accuracy = 0.968434362257

TRACE: trace2
size = 31788
accuracy = 0.981553796083

TRACE: trace3
size = 31788
accuracy = 0.990728583001

TRACE: trace4
size = 31788
accuracy = 0.990681392478

TRACE: trace5
size = 31788
accuracy = 0.997605746209

average predictor accuracy = 0.985800776005
expected average accuracy = 0.9512

```

Figure 10: Accuracy for predictor size 32000

Accuracy outputs for different traces of each budget size are shown below:

Traces :	Accuracy:
Trace1	0.9594
Trace2	0.9728
Trace3	0.9900
Trace4	0.9883
Trace5	0.9848
Average	0.9790

Table 3: Accuracy for predictor size 9999

Traces :	Accuracy:
Trace1	0.9684
Trace2	0.9815
Trace3	0.9907
Trace4	0.9906
Trace5	0.9976
Average	0.9858

Table 4: Accuracy for predictor size 32000

3. WEKA) Tool - Overview & Outputs:

To compare the results obtained in the above implementation of various Branch predictors, I have used the WEKA toolkit. The WEKA tool uses a machine-learning algorithm to predict and analyse the outcomes. One of the screenshots of the WEKA-tool in action has been shown below:

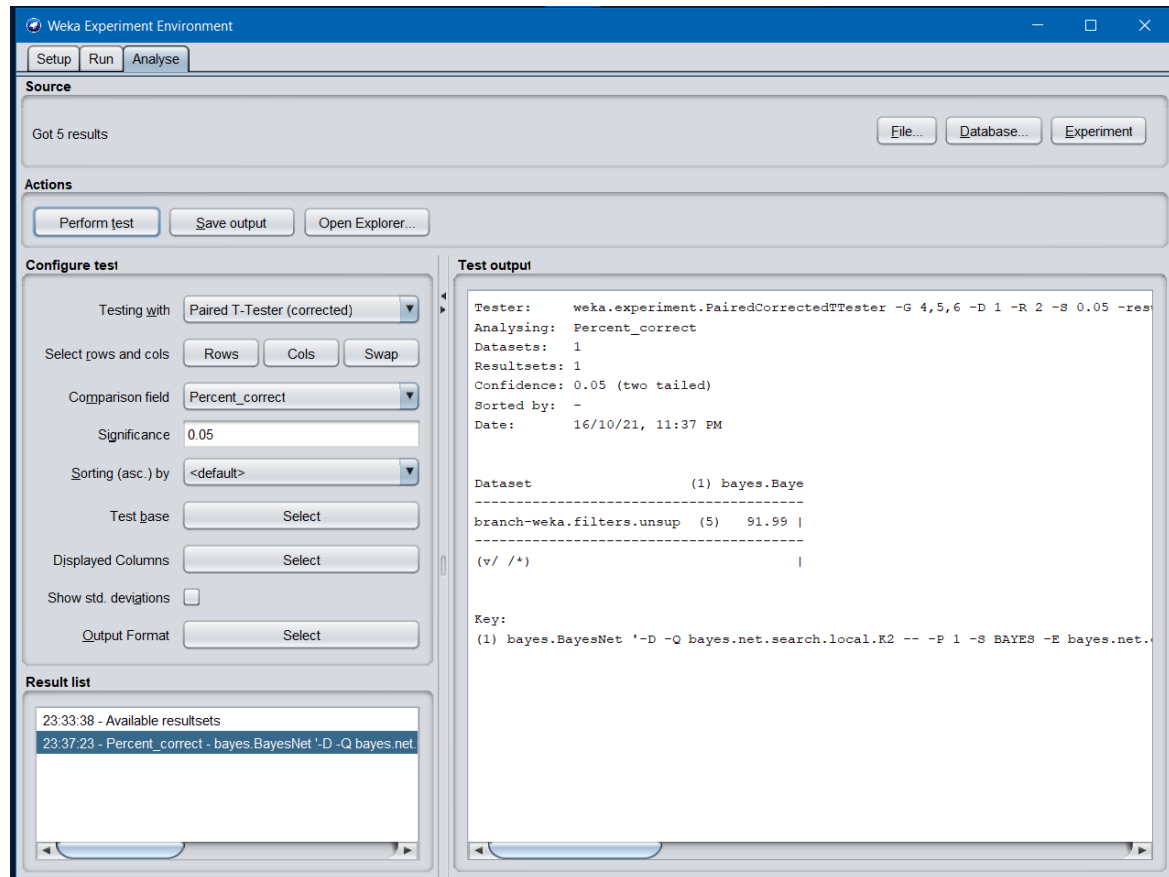


Figure 7: WEKA toolkit

For my comparison purpose, I have used Bayes-Net-classifier as a machine learning algorithm. I have run 5 folds cross-validation under Bayes-Net-Classifer. The Accuracy outputs, which I got for different traces by running both the WEKA tool are as follows:

Trace	WEKA - Accuracy
Trace1	91.99
Trace2	91.83
Trace3	98.66
Trace4	95.40
Trace5	79.83

Table 5: WEKA Accuracy outputs for different traces

3.1. Confusion Matrix & accuracy by WEKA-tool :

- Trace1:

Correctly Classified Instances	2036390	91.9915 %
Incorrectly Classified Instances	177283	8.0085 %

=== Confusion Matrix ===

	a	b	<-- classified as
1665197	135577		a = 0
41706	371193		b = 1

- Trace2:

Correctly Classified Instances	1646429	91.8338 %
Incorrectly Classified Instances	146406	8.1662 %

=== Confusion Matrix ===

	a	b	<-- classified as
1302518	123430		a = 0
22976	343911		b = 1

- Trace3:

Correctly Classified Instances	1526193	98.668 %
Incorrectly Classified Instances	20604	1.332 %

=== Confusion Matrix ===

	a	b	<-- classified as
169081	18508		a = 0
2096	1357112		b = 1

- Trace4:

Correctly Classified Instances	854681	95.4053 %
Incorrectly Classified Instances	41161	4.5947 %

=== Confusion Matrix ===

	a	b	<-- classified as
54181	38359		a = 0
2802	800500		b = 1

- Trace5:

Correctly Classified Instances	1933646	79.8351 %
Incorrectly Classified Instances	488403	20.1649 %

=== Confusion Matrix ===

	a	b	<-- classified as
575366	450369		a = 0
38034	1358280		b = 1

4. (TEJAS) Simulator – Overview & Operations:

Tejas is a Java-based multicore architectural simulator. It supports multicore simulation, cache coherence, NUCA protocols, and power simulation. It generates analysis output which includes the number of handled CISC instructions, Total IPC, branch predictor accuracy etc.

Later, the timing simulator generates a custom instruction set - VISA (Virtual Instruction Set Architecture). Then simulation of pipelines is done. As soon as the simulation gets completed, all the detailed statistics regarding the IPCs, caches, and cores are produced at the output.

In Tejas-tool, TAGE has been used to simulate branch predictors. The number of PC-bits, BHR-size and saturating counter- bits used for the TAGE predictor is shown in the figure below. Simulation for Out Of Order pipeline has been carried out. The simulation result indicates that in OOO pipelines there is a decrease in IPC with the increase in Core-frequency.

The working of Tejas-tool has been summarized in the table below:

```
Pipeline: outOfOrder
instructions executed   =      970826
cycles taken          =    2239909 cycles
IPC                   =           0.4334      in terms of micro-ops
IPC                   =           0.4224      in terms of CISC instructions
core frequency        =      3200 MHz
time taken            =      699.9716 microseconds

number of branches     =      331478
number of mispredicted branches =      14532
branch predictor accuracy =              95.6160 %

predictor type = TAGE
PC bits = 8
BHR size = 16
Saturating bits = 2
```

Figure 8: Simulation Output of TEJAS tool

Branch Predictor	TAGE
Branch Predictor Accuracy	95.61 %
Pipeline	Out Of Order
Core Frequency (MHz)	3200
IPC (CISC instructions)	0.4224

Table 6: Simulator – Summary of Tejas tool